DEPARTMENT OF SOFTWARE ENGINERING (DEGREE PROGRAM)

Database Group Assignment

COURSE TITLE: Database **SECTION A**
Course code: **Group 6**

Assignment TITLE-STUDENT MANAGEMENT SYSTEM

NAME	ID NUMBER
1 . Addis Tegene	ETS0097/17
2. Amanuel Alemayehu	ETS0144/17
3. Amensisa Geremew	ETS0169/17
4. Abel Gebregziabher	ETS0028/17
5. Birhanu Liku	ETS0321/17
6.Dawit Woldtinsae	ETS0428/17

Submitted to:instructor yaynshet medhin
Submission Date:May 15/ 2025

FINAL PROJECT REPORT

STUDENT MANAGEMENT SYSTEM
(Database Design and Implementation using MySQL and MongoDB)

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

There is a lot of information kept at Addis Ababa Science and Technology University including student data, course enrollments, teacher assignments, among others. As the number of both the student body and the number of courses offered increases, the problem with using the traditional or manual way of managing all this information arises. Such ways involve difficulties in terms of redundancy of data, inconsistencies in the same, and ineffective search for the needed information. Therefore, there is a need for creating an effective system which will help keep and manage the information more effectively.
It is in this respect that a student management system which uses MySQL and MongoDB databases has been designed and implemented.

1.2 Problem Statement

The existing system of managing student information faces the following issues:
Repetition of data within records
Inefficiency in updating and maintaining consistency
Problems in searching and retrieving student and course information
Inefficiency in managing relationships among students, courses, and teachers
Thus, a need exists for an effective database system to resolve these problems.

1.3 Objectives of the Study

General Objective
To design and implement a Student Management System using MySQL and MongoDB.
Specific Objectives
To store student, course, instructor, and department data efficiently
To manage student course enrollment
To reduce data redundancy using normalization
To implement relational and NoSQL databases
To demonstrate basic database operations

1.4 Scope of the Study

This project covers the design and implementation of a Student Management System including:
Student registration
Course management
Instructor assignment
Department management
Student enrollment and grading
It does not include a graphical user interface or online system deployment.

1.5 Significance of the Study

This system improves data organization and ensures efficient storage and retrieval of academic records. It also helps students understand how relational and NoSQL databases are applied in real-world systems.

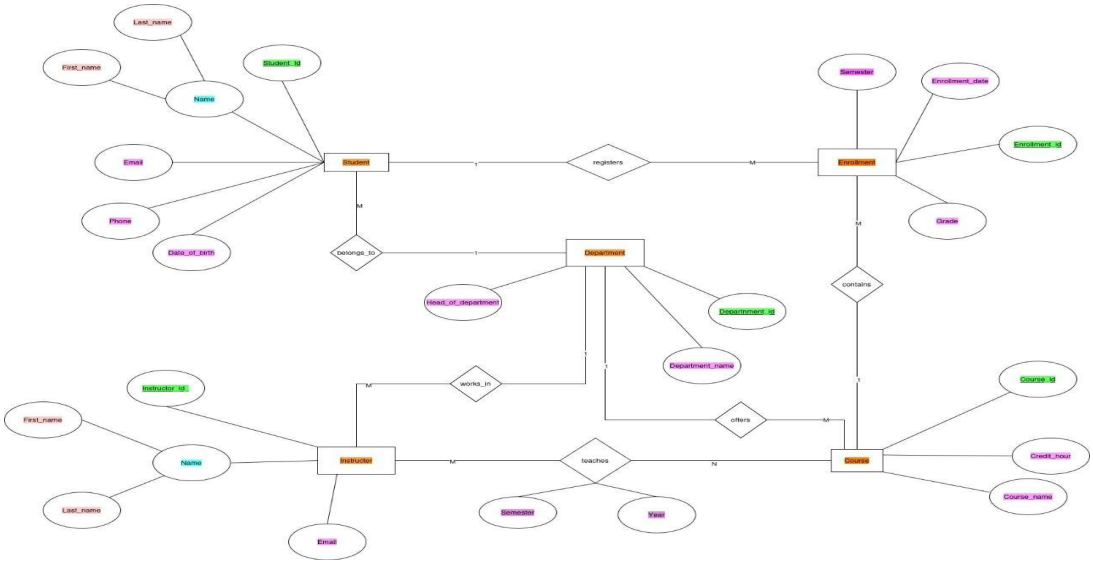
1.6 Methodology and Tools

The project follows a structured approach:
Requirement analysis
ER diagram design
Database normalization
Implementation
Tools used:
MySQL (Relational Database)
MongoDB (NoSQL Database)
Draw.io (ER Diagram)
JSON & SQL for data handling

CHAPTER TWO: DATABASE DESIGN

2.1 Identified Entities

The system consists of the following entities:
Student
Course
Instructor
Department
Enrollment



2.2 Attributes of Entities

Student
student_id (PK)
first_name
last_name
email
phone
date_of_birth
department_id (FK)
Course
course_id (PK)
course_name
credit_hour
instructor_id (FK)
Instructor
instructor_id (PK)
first_name
last_name
email
department_id (FK)
Department
department_id (PK)
department_name
Enrollment
enrollment_id (PK)
student_id (FK)
course_id (FK)
enrollment_date
grade

2.3 Relationships

Department → Student (1:M)
Department → Instructor (1:M)
Instructor → Course (1:M)

Student ↔ Course (M:N via Enrollment)
2.4 Relational Schema (MySQL Design)

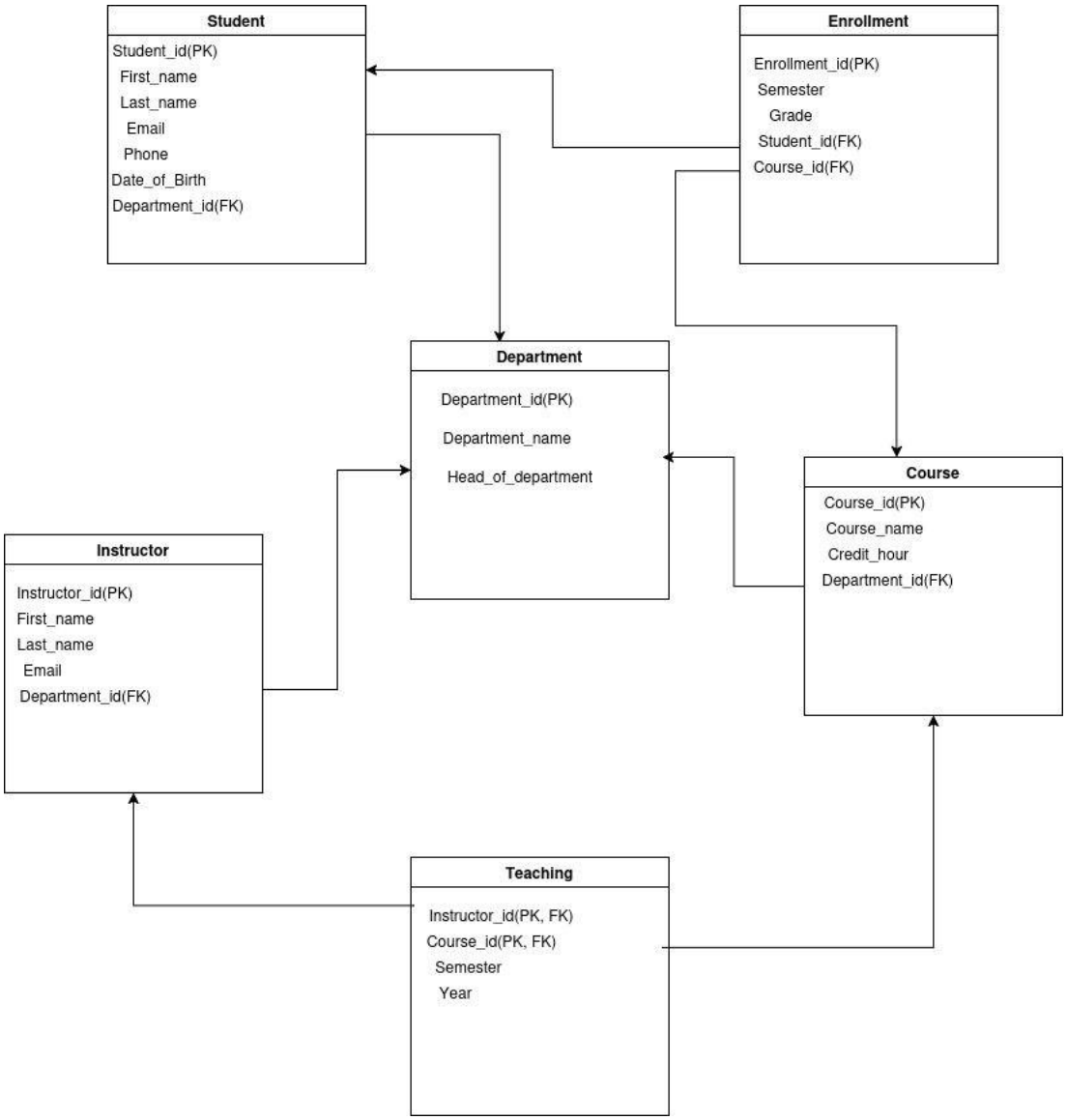
Student(student_id, first_name, last_name, email, phone, date_of_birth, department_id)

Course(course_id, course_name, credit_hour, instructor_id)

Instructor(instructor_id, first_name, last_name, email, department_id)

Department(department_id, department_name)

Enrollment(enrollment_id, student_id, course_id, enrollment_date, grade)



2.5 Normalization

Step 1: Unnormalized Form (UNF)

In the beginning, all data may exist in one large table containing repeating groups and redundant information.

UNF Table

student_id	student_name	department_name	course_id	course_name	instructor_name	enrollment_date	grade
101	Abel Tadesse	Computer Science	C101	Database Systems	Dr. Bekele	2026-01-10	A
101	Abel Tadesse	Computer Science	C102	Networking	Dr. Hana	2026-01-10	B
102	Meron Ali	Software Engineering	C101	Database Systems	Dr. Bekele	2026-01-11	A

Problems in UNF

- Repeating course records for one student
- Data redundancy
- Difficult update operations
- Insertion and deletion anomalies
- Poor data consistency

First Normal Form (1NF)

A table is in **1NF** when:

- All attributes contain atomic (single) values
- No repeating groups exist
- Each row is unique

1NF Relation

enrollment_id	student_id	student_name	department_name	course_id	course_name	instructor_name	enrollment_date	grade
E001	101	Abel Tadesse	Computer Science	C101	Database Systems	Dr. Bekele	2026-01-10	A
E002	101	Abel Tadesse	Computer Science	C102	Networking	Dr. Hana	2026-01-10	B
E003	102	Meron Ali	Software Engineering	C101	Database Systems	Dr. Bekele	2026-01-11	A

Improvements Achieved

- Repeating groups removed
- Atomic values ensured
- Easier searching and sorting

Remaining Problems

- Partial dependency still exists
- Data redundancy remains
- Update anomalies still possible

Second Normal Form (2NF)

A table is in **2NF** when:

- It is already in 1NF
- All non-key attributes fully depend on the entire primary key
- No partial dependency exists

To achieve 2NF, the large table is decomposed into separate relations.

STUDENT Table

student_id (PK)	first_name	last_name	email	phone	date_of_birth	department_id
101	Abel	Tadesse	abel@gmail.com	0911223344	2003-04-12	D01
102	Meron	Ali	meron@gmail.com	0911334455	2002-08-21	D02

DEPARTMENT Table

department_id (PK)	department_name
D01	Computer Science
D02	Software Engineering

INSTRUCTOR Table

instructor_id (PK)	first_name	last_name	email	department_id
I01	Bekele	Alemu	bekele@aastu.edu.et	D01
I02	Hana	Tesfaye	hana@aastu.edu.et	D02

COURSE Table

course_id (PK)	course_name	credit_hour	instructor_id
C101	Database Systems	3	I01
C102	Networking	4	I02

ENROLLMENT Table

enrollment_id (PK)	student_id (FK)	course_id (FK)	enrollment_date	grade
E001	101	C101	2026-01-10	A
E002	101	C102	2026-01-10	B

enrollment_id (PK)	student_id (FK)	course_id (FK)	enrollment_date	grade
E003	102	C101	2026-01-11	A

Improvements Achieved

- Partial dependencies removed
- Reduced redundancy
- Better organization of data

Remaining Problem

- Possible transitive dependencies

Third Normal Form (3NF)

A relation is in **3NF** if:

- It is already in 2NF
- No transitive dependency exists
- Non-key attributes depend only on the primary key

3NF Tables

1. DEPARTMENT Table (3NF)

department_id (PK)	department_name
1	Computer Science
2	Software Engineering
3	Information Technology

2. STUDENT Table (3NF)

student_id (PK)	first_name	last_name	email	phone	date_of_birth	department_id (FK)
101	Abel	Tadesse	abel@gmail.com	0911223344	2003-04-12	1

student_id (PK)	first_name	last_name	email	phone	date_of_birth	department_id (FK)
102	Meron	Ali	meron@gmail.com	0911334455	2002-08-21	2

3. INSTRUCTOR Table (3NF)

instructor_id (PK)	first_name	last_name	email	department_id (FK)
201	Bekele	Alemu	bekele@aastu.edu.et	1
202	Hana	Tesfaye	hana@aastu.edu.et	2

4. COURSE Table (3NF)

course_id (PK)	course_name	credit_hour	instructor_id (FK)
301	Database Systems	3	201
302	Networking	4	202

5. ENROLLMENT Table (3NF)

enrollment_id (PK)	student_id (FK)	course_id (FK)	enrollment_date	grade
401	101	301	2026-01-10	A
402	101	302	2026-01-10	B
403	102	301	2026-01-11	A

Boyce-Codd Normal Form (BCNF)

A relation is in **BCNF** if:

- For every functional dependency:

X→YX \rightarrow YX→Y

- X must be a candidate key.

BCNF Tables

1. DEPARTMENT Table (BCNF)

department_id (PK)	department_name
1	Computer Science
2	Software Engineering
3	Information Technology

Functional Dependency:

department_id→department_namedepartment_id \rightarrow department_namedepartment_id→department_name

2. STUDENT Table (BCNF)

student_id (PK)	first_name	last_name	email	phone	date_of_birth	department_id (FK)
101	Abel	Tadesse	abel@gmail.com	0911223344	2003-04-12	1
102	Meron	Ali	meron@gmail.com	0911334455	2002-08-21	2

Functional Dependency:

student_id→first_name, last_name, email, phone, date_of_birth, department_idstudent_id \rightarrow first_name,\ last_name,\ email,\ phone,\ date_of_birth,\ department_idstudent_id→first_name, last_name, email, phone, date_of_birth, department_id

3. INSTRUCTOR Table (BCNF)

instructor_id (PK)	first_name	last_name	email	department_id (FK)

instructor_id (PK)	first_name	last_name	email	department_id (FK)
201	Bekele	Alemu	bekele@aastu.edu.et	1
202	Hana	Tesfaye	hana@aastu.edu.et	2

Functional Dependency:

instructor_id→first_name, last_name, email, department_idinstructor_id \rightarrow first_name,\ last_name,\ email,\ department_idinstructor_id→first_name, last_name, email, department_id

4. COURSE Table (BCNF)

course_id (PK)	course_name	credit_hour	instructor_id (FK)
301	Database Systems	3	201
302	Networking	4	202

Functional Dependency:

course_id→course_name, credit_hour, instructor_idcourse_id \rightarrow course_name,\ credit_hour,\ instructor_idcourse_id→course_name, credit_hour, instructor_id

5. ENROLLMENT Table (BCNF)

enrollment_id (PK)	student_id (FK)	course_id (FK)	enrollment_date	grade
401	101	301	2026-01-10	A
402	101	302	2026-01-10	B
403	102	301	2026-01-11	A

Functional Dependency:

enrollment_id→student_id, course_id, enrollment_date, gradeenrollment_id \rightarrow student_id,\ course_id,\ enrollment_date,\ gradeenrollment_id→student_id, course_id, enrollment_date, grade

2.7 Input Design (Forms)

Input design refers to the forms used to collect and enter data into the Student Management System. Proper input design improves accuracy, consistency, and ease of use.

1. Student Registration Form

Purpose

Used to register new students into the system.

Fields Included

Field Name	Data Type	Description
Student ID	Integer	Unique student identifier
First Name	Text	Student first name
Last Name	Text	Student last name
Email	Text	Student email address
Phone Number	Text	Student contact number
Date of Birth	Date	Student birth date
Department	Dropdown List	Department selection

2. Course Registration Form

Purpose

Used to add and manage course information.

Fields Included

Field Name	Data Type	Description
Course ID	Integer	Unique course identifier
Course Name	Text	Name of the course
Credit Hour	Integer	Number of credit hours
Instructor ID	Dropdown List	Assigned instructor

3. Instructor Registration Form

Purpose

Used to register instructors in the system.

Fields Included

Field Name	Data Type	Description
Instructor ID	Integer	Unique instructor identifier
First Name	Text	Instructor first name
Last Name	Text	Instructor last name
Email	Text	Instructor email
Department	Dropdown List	Assigned department

4. Department Form

Purpose

Used to manage department information.

Fields Included

Field Name	Data Type	Description
Department ID	Integer	Unique department identifier
Department Name	Text	Name of department

5. Enrollment Form

Purpose

Used to enroll students into courses.

Fields Included

Field Name	Data Type	Description
Enrollment ID	Integer	Unique enrollment identifier

Field Name	Data Type	Description
Student ID	Dropdown List	Selected student
Course ID	Dropdown List	Selected course
Enrollment Date	Date	Date of enrollment
Grade	Text	Student grade

2.8 Output Design

Output design refers to the information produced by the system after processing data. Outputs help users retrieve and analyze important information.

1. Student Information Output

Description

Displays detailed information about registered students.

Output Fields

Student ID	Student Name	Department	Email	Phone
101	Abel Tadesse	Computer Science	abel@gmail.com	0911223344

2. Course Information Output

Description

Displays all available courses and assigned instructors.

Output Fields

Course ID	Course Name	Credit Hour	Instructor
301	Database Systems	3	Bekele Alemu

3. Enrollment Information Output

Description

Displays students enrolled in courses.

Output Fields

Enrollment ID	Student Name	Course Name	Enrollment Date	Grade
401	Abel Tadesse	Database Systems	2026-01-10	A

4. Instructor Information Output

Description

Displays instructor details and their departments.

Output Fields

Instructor ID	Instructor Name	Department	Email
201	Bekele Alemu	Computer Science	bekele@aastu.edu.et

2.9 Report Design

Reports summarize processed information for decision-making and monitoring.

1. Student Report

Purpose

Provides a complete list of students registered in the system.

Report Contents

- Student ID
- Full Name
- Department
- Contact Information

2. Course Report

Purpose

Displays all courses offered by the institution.

Report Contents

- Course ID
- Course Name
- Credit Hour
- Assigned Instructor

3. Enrollment Report

Purpose

Shows students enrolled in different courses.

Report Contents

- Student Name
- Course Name
- Enrollment Date
- Grade

4. Department Report

Purpose

Displays departments and associated students/instructors.

Report Contents

- Department Name
- Number of Students
- Number of Instructors

5. Academic Performance Report

Purpose

Used to evaluate student academic performance.

Report Contents

- Student Name
- Course Name
- Grade
- GPA/Result Summary

CHAPTER THREE: IMPLEMENTATION

3.1 MySQL Implementation

The Student Management System was implemented using MySQL relational database management system. The implementation includes:

- Database creation
- Table creation
- Primary and foreign key constraints
- Data insertion
- Queries
- Reports

```
CREATE DATABASE StudentManagementSystem;

USE StudentManagementSystem;

CREATE TABLE Department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(100)
);

CREATE TABLE Instructor (
    instructor_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    department_id INT,
    FOREIGN KEY (department_id)
    REFERENCES Department(department_id)
```

```
);

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    phone VARCHAR(20),
    date_of_birth DATE,
    department_id INT,
    FOREIGN KEY (department_id)
    REFERENCES Department(department_id)
);

CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    credit_hour INT,
    instructor_id INT,
    FOREIGN KEY (instructor_id)
    REFERENCES Instructor(instructor_id)
);

CREATE TABLE Enrollment (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    grade VARCHAR(5),
    FOREIGN KEY (student_id)
    REFERENCES Student(student_id),
    FOREIGN KEY (course_id)
    REFERENCES Course(course_id)
);

INSERT INTO Department VALUES
(1, 'food science'),
(2, 'Software Engineering'),
(3, 'environmental enginnering'),
(4, 'electrical enginnering');

INSERT INTO Instructor VALUES
(201, 'Bekele', 'Alemu', 'bekele@aastu.edu.et', 1),
(202, 'Hana', 'Tesfaye', 'hana@aastu.edu.et', 2),
(203, 'Samuel', 'Taye', 'samuel@aastu.edu.et', 3),
(204, 'Rahel', 'Kebede', 'rahel@aastu.edu.et', 4);

INSERT INTO Student VALUES
(101, 'Abel', 'Tadesse', 'abel@gmail.com', '0911223344', '2003-04-12', 1),
(102, 'Meron', 'Ali', 'meron@gmail.com', '0911334455', '2002-08-21', 2),
```

```
(103, 'Dawit', 'Kassa', 'dawit@gmail.com', '0911445566', '2001-07-10', 1),
(104, 'Bethel', 'Mulu', 'bethel@gmail.com', '0911556677', '2004-01-17', 3),
(105, 'Nahom', 'Teklu', 'nahom@gmail.com', '0911667788', '2002-05-25', 4);

INSERT INTO Course VALUES
(301, 'Database Systems', 3, 201),
(302, 'Networking', 4, 202),
(303, 'Operating Systems', 3, 203),
(304, 'Machine Learning', 4, 204);

INSERT INTO Enrollment VALUES
(401, 101, 301, '2026-01-10', 'A'),
(402, 101, 302, '2026-01-10', 'B'),
(403, 102, 301, '2026-01-11', 'A'),
(404, 103, 303, '2026-01-12', 'B+'),
(405, 104, 304, '2026-01-13', 'A-'),
(406, 105, 302, '2026-01-14', 'B');

SELECT * FROM Student;

SELECT * FROM Course;

SELECT * FROM Instructor;

SELECT * FROM Department;

SELECT * FROM Enrollment;

SELECT
   Student.student_id,
   Student.first_name,
   Student.last_name,
   Department.department_name
FROM Student
JOIN Department
ON Student.department_id = Department.department_id;

SELECT
   Course.course_id,
   Course.course_name,
   Course.credit_hour,
   Instructor.first_name,
   Instructor.last_name
FROM Course
JOIN Instructor
ON Course.instructor_id = Instructor.instructor_id;

SELECT
   Student.first_name,
   Student.last_name,
```

```
    Course.course_name,
    Enrollment.grade
FROM Enrollment
JOIN Student
ON Enrollment.student_id = Student.student_id
JOIN Course
ON Enrollment.course_id = Course.course_id;

SELECT COUNT(*) AS Total_Students
FROM Student;

SELECT COUNT(*) AS Total_Courses
FROM Course;

UPDATE Student
SET phone = '0911998877'
WHERE student_id = 101;

DELETE FROM Enrollment
WHERE enrollment_id = 406;

SELECT *
FROM Student
WHERE student_id = 102;

SELECT *
FROM Course
WHERE course_name = 'Database Systems';

SELECT
    Student.first_name,
    Student.last_name,
    Course.course_name,
    Enrollment.grade
FROM Enrollment
JOIN Student
ON Enrollment.student_id = Student.student_id
JOIN Course
ON Enrollment.course_id = Course.course_id
WHERE Enrollment.grade = 'A';

SELECT *
FROM Student
ORDER BY first_name ASC;

SELECT *
FROM Course
WHERE credit_hour > 3;
```

schema.sql

```
CREATE DATABASE StudentManagementSystem;

USE StudentManagementSystem;

CREATE TABLE Department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(100)
);

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    department_id INT,
    FOREIGN KEY (department_id)
    REFERENCES Department(department_id)
);

CREATE TABLE Instructor (
    instructor_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department_id INT,
    FOREIGN KEY (department_id)
    REFERENCES Department(department_id)
);

CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    credit_hour INT,
    instructor_id INT,
    FOREIGN KEY (instructor_id)
    REFERENCES Instructor(instructor_id)
);

CREATE TABLE Enrollment (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    grade VARCHAR(5),
    FOREIGN KEY (student_id)
    REFERENCES Student(student_id),
    FOREIGN KEY (course_id)
    REFERENCES Course(course_id)
);
```

3.2 MongoDB Implementation

The Student Management System was also implemented using MongoDB, which is a NoSQL document-oriented database. MongoDB stores data in collections and documents using JSON-like format.

The implementation includes:

- Creating collections
- Inserting documents
- Retrieving documents
- Updating documents
- Deleting documents
- Query operations

```
use StudentManagementSystem

db.Department.insertMany([
{
   department_id: 1,
   department_name: "food science"
},
{
   department_id: 2,
   department_name: "Software Engineering"
},
{
   department_id: 3,
   department_name: "environmental enginering"
},
{
   department_id: 4,
   department_name: "electrical "
}
])

db.Instructor.insertMany([
{
   instructor_id: 201,
   first_name: "Bekele",
   last_name: "Alemu",
   email: "bekele@aastu.edu.et",
   department_id: 1
},
{
   instructor_id: 202,
   first_name: "Hana",
   last_name: "Tesfaye",
   email: "hana@aastu.edu.et",
   department_id: 2
},
{
   instructor_id: 203,
```

```
    first_name: "Samuel",
    last_name: "Taye",
    email: "samuel@aastu.edu.et",
    department_id: 3
},
{
    instructor_id: 204,
    first_name: "Rahel",
    last_name: "Kebede",
    email: "rahel@aastu.edu.et",
    department_id: 4
}
])

db.Student.insertMany([
{
    student_id: 101,
    first_name: "Abel",
    last_name: "Tadesse",
    email: "abel@gmail.com",
    phone: "0911223344",
    date_of_birth: "2003-04-12",
    department_id: 1
},
{
    student_id: 102,
    first_name: "Meron",
    last_name: "Ali",
    email: "meron@gmail.com",
    phone: "0911334455",
    date_of_birth: "2002-08-21",
    department_id: 2
},
{
    student_id: 103,
    first_name: "Dawit",
    last_name: "Kassa",
    email: "dawit@gmail.com",
    phone: "0911445566",
    date_of_birth: "2001-07-10",
    department_id: 1
},
{
    student_id: 104,
    first_name: "Bethel",
    last_name: "Mulu",
    email: "bethel@gmail.com",
    phone: "0911556677",
    date_of_birth: "2004-01-17",
    department_id: 3
```

```
},
{
  student_id: 105,
  first_name: "Nahom",
  last_name: "Teklu",
  email: "nahom@gmail.com",
  phone: "0911667788",
  date_of_birth: "2002-05-25",
  department_id: 4
}
])

db.Course.insertMany([
{
  course_id: 301,
  course_name: "Database Systems",
  credit_hour: 3,
  instructor_id: 201
},
{
  course_id: 302,
  course_name: "Networking",
  credit_hour: 4,
  instructor_id: 202
},
{
  course_id: 303,
  course_name: "Operating Systems",
  credit_hour: 3,
  instructor_id: 203
},
{
  course_id: 304,
  course_name: "Machine Learning",
  credit_hour: 4,
  instructor_id: 204
}
])

db.Enrollment.insertMany([
{
  enrollment_id: 401,
  student_id: 101,
  course_id: 301,
  enrollment_date: "2026-01-10",
  grade: "A"
},
{
  enrollment_id: 402,
  student_id: 101,
```

```
    course_id: 302,
    enrollment_date: "2026-01-10",
    grade: "B"
},
{
    enrollment_id: 403,
    student_id: 102,
    course_id: 301,
    enrollment_date: "2026-01-11",
    grade: "A"
},
{
    enrollment_id: 404,
    student_id: 103,
    course_id: 303,
    enrollment_date: "2026-01-12",
    grade: "B+"
},
{
    enrollment_id: 405,
    student_id: 104,
    course_id: 304,
    enrollment_date: "2026-01-13",
    grade: "A-"
},
{
    enrollment_id: 406,
    student_id: 105,
    course_id: 302,
    enrollment_date: "2026-01-14",
    grade: "B"
}
])

db.Student.find()

db.Course.find()

db.Instructor.find()

db.Department.find()

db.Enrollment.find()

db.Student.find(
{
    student_id: 101
}
)
```

```
db.Course.find(
{
   course_name: "Database Systems"
}
)

db.Enrollment.find(
{
   grade: "A"
}
)

db.Student.updateOne(
{
   student_id: 101
},
{
   $set:
   {
      phone: "0911998877"
   }
}
)

db.Enrollment.deleteOne(
{
   enrollment_id: 406
}
)

db.Student.find().sort(
{
   first_name: 1
}
)

db.Course.find(
{
   credit_hour:
   {
      $gt: 3
   }
}
)

db.Student.countDocuments()

db.Course.countDocuments()
```

Collection.json

```
{
 "Department": [
   {
    "department_id": 1,
    "department_name": "Computer Science"
   },
   {
    "department_id": 2,
    "department_name": "Software Engineering"
   }
 ],

 "Student": [
   {
    "student_id": 101,
    "first_name": "Abel",
    "last_name": "Tadesse",
   "email": "abel@gmail.com",
    "department_id": 1
   },
   {
    "student_id": 102,
    "first_name": "Meron",
    "last_name": "Ali",
    "email": "meron@gmail.com",
    "department_id": 2
   }
 ],

 "Instructor": [
   {
    "instructor_id": 201,
    "first_name": "Bekele",
    "last_name": "Alemu",
    "department_id": 1
   }
 ],

 "Course": [
   {
    "course_id": 301,
    "course_name": "Database Systems",
    "credit_hour": 3,
    "instructor_id": 201
   }
 ],

 "Enrollment": [
   {
```

```
    "enrollment_id": 401,
    "student_id": 101,
    "course_id": 301,
    "grade": "A"
   }
  ]
}
```


3.3 Testing

The Student Management System was tested after implementation in both MySQL and MongoDB environments. Different operations such as insertion, retrieval, updating, deletion, and searching were performed to verify the correctness of the system.

Testing Activities Performed

Test Case	Operation Performed	Expected Result	Actual Result	Status
1	Insert student record	Student record stored successfully	Record inserted correctly	Passed
2	Insert course record	Course saved in database	Course stored successfully	Passed
3	Enroll student in course	Enrollment created	Enrollment inserted correctly	Passed
4	Search student by ID	Student information displayed	Correct student displayed	Passed
5	Update student email	New email updated	Email updated successfully	Passed
6	Delete enrollment record	Enrollment removed	Record deleted correctly	Passed
7	Display all students	All student records shown	Records displayed successfully	Passed
8	Count total students	Correct number displayed	Correct count returned	Passed
9	Retrieve courses with credit hour > 3	Matching courses displayed	Correct courses displayed	Passed
10	Retrieve students with grade A	Students with grade A shown	Correct result displayed	Passed

Results

The testing results showed that the system functions correctly and satisfies the project objectives.

The system successfully:

- Stores student, course, instructor, department, and enrollment data
- Retrieves records accurately
- Updates records without affecting related data
- Deletes records correctly
- Reduces redundancy through normalization
- Maintains data consistency and integrity
- Performs database operations efficiently in both MySQL and MongoDB

The implemented database structure also supports easy searching, sorting, and reporting of academic information.

Bugs Encountered

During implementation and testing, several minor issues were encountered:

Bug	Description	Solution
Foreign key constraint error	Enrollment inserted before student/course existed	Insert parent records first
Duplicate primary key error	Same ID inserted multiple times	Use unique IDs
Typing errors in queries	Incorrect table or column names used	Corrected query syntax
Missing field values	Some records inserted with incomplete data	Added required fields
Incorrect data type	String inserted into integer field	Corrected data types

Most bugs were syntax-related and were resolved during testing.

Improvements

The following improvements are recommended for future development of the system:

1. Develop a graphical user interface (GUI)

2. Create a web-based version of the system
3. Add login and authentication features
4. Implement advanced report generation
5. Add automatic backup and recovery mechanisms
6. Improve data validation and security
7. Add student GPA calculation functionality
8. Integrate the system with online registration services
9. Implement role-based access control for administrators, instructors, and students
10. Expand MongoDB implementation with advanced aggregation queries

Survey Questionnaire

Student Management System Requirement Analysis Questionnaire

Addis Ababa Science and Technology University

Section A: General Information

1. What is your role in the institution?

- Student
- Instructor
- Administrator

1. Which department do you belong to?

Section B: Current System Evaluation

1. How do you currently manage student information?

- Manual paper records
- Excel sheets
- Existing software system
- Other _____

1. What problems do you face in the current system?

- Data duplication
- Slow retrieval of information
- Difficulty updating records
- Data inconsistency
- Loss of records

1. How often do errors occur during data handling?

- Frequently
- Sometimes
- Rarely
- Never

Section C: System Requirements

1. Do you think a computerized database system is necessary?

- Yes
- No

1. Which features should the new system include?

- Student registration
- Course management
- Enrollment management
- Grade tracking
- Report generation

1. Should the system support fast searching and retrieval of records?

- Yes
- No

Section D: Suggestions

1. What improvements would you like to see in the new system?

Sample Forms

1. Student Registration Form

Student ID : _____

First Name : _____

Last Name : _____

Email : _____

Phone Number : _____

Date of Birth : _____

Department : _____

2. Course Registration Form

Course ID : _____

Course Name : _____

Credit Hour : _____

Instructor ID : _____

3. Instructor Registration Form

Instructor ID : _____

First Name : _____

Last Name : _____

Email : _____

Department ID : _____

4. Enrollment Form

Enrollment ID : _____

Student ID : _____

Course ID : _____

Enrollment Date : _____

Grade : _____